

Penjelasan Lengkap Project CoretDraw

Dokumen ini menjelaskan cara kerja project CoretDraw secara lengkap, mulai dari workflow aplikasi, struktur modul, fitur yang tersedia, sampai algoritma grafika komputer yang digunakan.

CoretDraw adalah aplikasi grafika 2D interaktif berbasis Python dan PySide6. Aplikasi ini menyediakan canvas untuk menggambar objek 2D, memilih dan memindahkan objek, memberi warna, melakukan transformasi, menjalankan animasi sederhana, undo/redo, serta export hasil gambar ke file PNG.

1. Gambaran Umum Aplikasi

CoretDraw berjalan sebagai aplikasi desktop Qt. Entry point aplikasi berada di file `main.py`.

Alur awal aplikasi:

1. Program membuat objek `QApplication`.
2. Program membuat window utama melalui class `MainWindow`.
3. `MainWindow` membuat `CanvasWidget`, `Sidebar`, status bar, menu bar, dan `Animator`.
4. Window ditampilkan dengan `window.show()`.
5. Event loop Qt dijalankan dengan `app.exec()`.

Secara sederhana, arsitektur aplikasi adalah:

```
main.py
-> MainWindow
    -> CanvasWidget
        -> Renderer
        -> HistoryManager
        -> Tool aktif
    -> Sidebar
        -> Mengubah tool, warna, ukuran, algoritma, transformasi
    -> Animator
        -> Mengubah objek terpilih secara berkala dengan QTimer
```

2. Struktur Folder dan Tanggung Jawab Modul

```
CoretDraw/
+-- main.py
+-- requirements.txt
+-- README.md
+-- PENJELASAN.md
+-- app/
    +-- algorithms/
    +-- animation/
    +-- canvas/
    +-- history/
    +-- models/
```

```
+-- tools/  
+-- transforms/  
+-- ui/
```

Penjelasan folder:

- `main.py`: entry point aplikasi.
- `app/ui/`: komponen antarmuka utama, yaitu `MainWindow` dan `Sidebar`.
- `app/canvas/`: area gambar dan renderer objek.
- `app/tools/`: tool interaksi pengguna seperti select, brush, eraser, fill, dan shape.
- `app/algorithms/`: implementasi algoritma grafika komputer.
- `app/models/`: model data objek grafis.
- `app/history/`: pengelola undo dan redo.
- `app/transforms/`: operasi transformasi objek.
- `app/animation/`: animasi sederhana berbasis timer.

3. Dependency

Project ini menggunakan dependency utama:

```
PySide6>=6.6.0
```

PySide6 digunakan untuk:

- Membuat window desktop.
- Membuat canvas dengan `QWidget`.
- Menggambar objek dengan `QPainter`.
- Mengatur warna dengan `QColor`.
- Membuat dialog, menu, tombol, slider, combo box, dan status bar.
- Menjalankan animasi dengan `QTimer`.

4. Workflow Menjalankan Aplikasi

Cara menjalankan project:

```
cd "D:\Semester 6\Grafkom\CoretDraw"  
python main.py
```

Jika dependency belum terpasang:

```
python -m pip install -r requirements.txt
```

Setelah aplikasi berjalan:

1. Window utama muncul.
2. Sidebar berada di sisi kiri.
3. Canvas putih berada di sisi kanan.
4. Pengguna memilih tool atau shape.
5. Pengguna menggambar atau memanipulasi objek di canvas.
6. Status bar menampilkan informasi posisi kursor atau status tertentu.

5. Komponen Utama

5.1 MainWindow

File: `app/ui/main_window.py`

`MainWindow` adalah window utama aplikasi. Class ini bertugas:

- Membuat judul window.
- Mengatur ukuran awal window.
- Membuat canvas.
- Membuat sidebar.
- Membuat status bar.
- Membuat menu File, Edit, Animation, dan Help.
- Menjalankan fungsi export PNG.
- Menampilkan dialog About dan Formula Transformasi.

Menu yang tersedia:

- `File > New`: membersihkan canvas.
- `File > Export PNG`: menyimpan canvas ke PNG.
- `File > Exit`: keluar dari aplikasi.
- `Edit > Undo`: membatalkan aksi terakhir.
- `Edit > Redo`: mengembalikan aksi yang dibatalkan.
- `Edit > Delete`: menghapus objek terpilih.
- `Animation > Move`: menjalankan animasi bergerak.
- `Animation > Rotate`: menjalankan animasi rotasi.
- `Animation > Scale`: menjalankan animasi skala.
- `Animation > Bounce`: menjalankan animasi pantul sederhana.
- `Animation > Stop`: menghentikan animasi.
- `Help > About`: menampilkan informasi aplikasi.
- `Help > Formula Transformasi`: menampilkan rumus transformasi.

5.2 Sidebar

File: `app/ui/sidebar.py`

Sidebar adalah panel kontrol di sebelah kiri canvas. Sidebar mengatur state yang digunakan oleh canvas dan tool.

Bagian sidebar:

- `TOOLS`: Select, Brush, Eraser, Fill.

- **SHAPES**: Point, Line, Circle, Ellipse, Rectangle, Triangle, Trapezoid.
- **ALGORITMA**: pilihan algoritma garis, lingkaran, dan elips.
- **WARNA**: pemilihan warna stroke dan fill.
- **UKURAN**: slider ketebalan garis atau ukuran brush.
- **TRANSFORMASI**: Translate, Rotate, Scale, Grayscale.
- **OBJEK**: Bring Front, Send Back, Delete.

Cara kerja sidebar:

1. Pengguna menekan tombol tool atau memilih shape.
2. Sidebar membuat instance tool yang sesuai.
3. Tool tersebut dikirim ke canvas melalui `canvas.set_tool(...)`.
4. Jika pengguna mengubah warna, ukuran, atau algoritma, sidebar langsung mengubah property pada canvas.
5. Jika ada objek terpilih, perubahan warna atau ukuran juga dapat diterapkan ke objek tersebut.

5.3 CanvasWidget

File: `app/canvas/canvas_widget.py`

CanvasWidget adalah area utama untuk menggambar. Class ini menyimpan semua state gambar dan menangani event mouse.

State penting pada canvas:

- **objects**: daftar semua objek grafis yang sudah dibuat.
- **preview_object**: objek sementara saat pengguna sedang drag shape.
- **selection_rect_preview**: kotak seleksi sementara bergaris putus-putus.
- **selected_objects**: daftar objek yang sedang dipilih.
- **stroke_color**: warna garis aktif.
- **fill_color**: warna isi aktif.
- **line_width**: ketebalan garis aktif.
- **line_algo**: algoritma garis aktif.
- **circle_algo**: algoritma lingkaran aktif.
- **ellipse_algo**: algoritma elips aktif.
- **_zoom**: nilai zoom canvas.
- **_offset_x** dan **_offset_y**: offset posisi tampilan.
- **_renderer**: objek **Renderer** untuk menggambar.
- **history**: objek **HistoryManager** untuk undo/redo.
- **_active_tool**: tool yang sedang aktif.

Workflow canvas:

1. Event mouse diterima oleh **CanvasWidget**.
2. Canvas meneruskan event ke tool aktif.
3. Tool aktif membuat atau memodifikasi **GraphicObject**.
4. Canvas memanggil `update()`.
5. Qt memanggil `paintEvent()`.

6. `paintEvent()` menggambar background putih, zoom, offset, lalu menggambar semua objek memakai `Renderer`.

5.4 Renderer

File: `app/canvas/renderer.py`

`Renderer` bertugas menggambar satu `GraphicObject` ke `QPainter`.

Renderer dipisahkan dari canvas agar:

- Canvas fokus pada event handling.
- Renderer fokus pada logika visualisasi.
- Algoritma gambar dipanggil di satu tempat.

Workflow render:

1. Canvas memanggil `renderer.draw(painter, obj)`.
2. Jika objek memiliki `image`, renderer menggambar `QImage` langsung.
3. Jika objek bukan image, renderer menyimpan state painter dengan `painter.save()`.
4. Renderer menerapkan transformasi objek:
 - translate ke pusat objek,
 - rotate,
 - scale,
 - translate kembali.
5. Renderer menentukan pen dan brush.
6. Renderer memanggil fungsi gambar sesuai `obj.kind`.
7. Painter dikembalikan dengan `painter.restore()`.
8. Jika objek selected, renderer menggambar kotak seleksi bergaris putus-putus.

Jenis objek yang didukung renderer:

- `point`
- `line`
- `circle`
- `ellipse`
- `rectangle`
- `triangle`
- `trapezoid`
- `brush`
- `eraser`
- objek berbasis `image`, misalnya hasil fill

6. Model Data GraphicObject

File: `app/models/graphic_object.py`

Setiap objek yang digambar di canvas direpresentasikan sebagai `GraphicObject`.

Property penting:

- `kind`: jenis objek, misalnya `line`, `circle`, `brush`, atau `fill`.
- `points`: daftar titik utama objek.
- `stroke`: warna garis.
- `fill`: warna isi.
- `width`: ketebalan garis.
- `line_algo`: algoritma garis yang dipakai.
- `circle_algo`: algoritma lingkaran yang dipakai.
- `ellipse_algo`: algoritma elips yang dipakai.
- `rotation`: sudut rotasi objek.
- `scale_x`: skala horizontal.
- `scale_y`: skala vertikal.
- `selected`: status apakah objek sedang dipilih.
- `image`: gambar bitmap untuk objek tertentu, misalnya `fill`.
- `path`: `QPainterPath` untuk `brush` dan `eraser`.

Method penting:

- `center()`: menghitung titik pusat dari bounding box objek.
- `bounding_rect()`: menghasilkan batas objek dalam format `(x_min, y_min, x_max, y_max)`.
- `move_by(dx, dy)`: memindahkan objek.
- `clone()`: membuat deep copy objek untuk undo/redo.

7. Sistem Tool

Semua tool mewarisi `BaseTool`.

File: `app/tools/base_tools.py`

Method wajib:

- `on_press(pos, event)`: dipanggil saat mouse ditekan.
- `on_move(pos, event)`: dipanggil saat mouse bergerak.
- `on_release(pos, event)`: dipanggil saat mouse dilepas.
- `on_double_click(pos, event)`: opsional.

Canvas tidak perlu tahu detail tiap tool. Canvas hanya meneruskan event mouse ke tool aktif. Dengan cara ini, setiap tool bisa memiliki logika sendiri.

8. Fitur Select

File: `app/tools/select_tool.py`

Tool Select digunakan untuk:

- Memilih objek dengan klik.
- Memindahkan objek dengan drag.
- Membuat kotak seleksi bergaris putus-putus seperti Windows Paint.

Workflow klik objek:

1. Pengguna klik canvas.

2. `SelectTool` menjalankan `_hit_test(pos)`.
3. `_hit_test` memeriksa objek dari urutan paling atas ke bawah.
4. Jika posisi klik berada di bounding box objek, objek dianggap terkena klik.
5. Objek tersebut diberi `selected = True`.
6. Objek lain di-deselect.
7. Canvas di-update.

Workflow drag objek:

1. Pengguna klik objek.
2. Tool menyimpan posisi awal ke `_last_pos`.
3. Saat mouse bergerak, tool menghitung:

```
dx = posisi_x_baru - posisi_x_lama  
dy = posisi_y_baru - posisi_y_lama
```

4. Semua objek terpilih dipindahkan dengan `obj.move_by(dx, dy)`.
5. Posisi terakhir diperbarui.
6. Canvas di-render ulang.

Workflow kotak seleksi:

1. Pengguna klik dan drag pada area kosong.
2. Tool menghapus seleksi lama.
3. Tool menyimpan titik awal drag sebagai `_selection_start`.
4. Canvas menyimpan `selection_rect_preview`.
5. Saat mouse bergerak, `selection_rect_preview` diperbarui.
6. Canvas menggambar rectangle preview dengan garis putus-putus.
7. Saat mouse dilepas, rectangle dinormalisasi.
8. Tool mengecek objek mana yang bounding box-nya bersinggungan dengan rectangle.
9. Objek yang bersinggungan akan dipilih.

Algoritma hit test:

- Menggunakan bounding box sederhana.
- Ada toleransi 8 pixel agar objek lebih mudah diklik.
- Urutan pengecekan dibalik, sehingga objek yang paling atas dipilih lebih dulu.

9. Fitur Brush

File: `app/tools/brush_tool.py`

Brush digunakan untuk menggambar bebas atau freehand.

Workflow:

1. Saat mouse ditekan, state canvas disimpan ke history.
2. Tool membuat `QPainterPath`.
3. Path dimulai dari posisi klik awal dengan `path.moveTo(pos)`.

4. Tool membuat `GraphicObject` dengan `kind="brush"`.
5. Objek brush ditambahkan ke canvas.
6. Saat mouse bergerak sambil tombol kiri ditekan, path ditambah dengan `lineTo(pos)`.
7. Titik baru juga disimpan ke `points`.
8. Saat mouse dilepas, path diselesaikan dan objek menjadi permanen.

Brush tidak menggunakan algoritma Bresenham atau DDA secara manual. Brush menggunakan `QPainterPath`, lalu renderer menggambarinya dengan `painter.drawPath(...)`.

10. Fitur Eraser

File: `app/tools/eraser_tool.py`

Eraser digunakan untuk menghapus secara visual.

Workflow:

1. Saat mouse ditekan, state canvas disimpan ke history.
2. Tool membuat `QPainterPath`.
3. Tool membuat `GraphicObject` dengan `kind="eraser"`.
4. Warna stroke eraser dibuat putih (`#ffffff`).
5. Ketebalan eraser dibuat lebih besar dari brush, yaitu `max(line_width * 3, 12)`.
6. Saat mouse bergerak, path bertambah seperti brush.
7. Renderer menggambar path putih di atas objek lain.

Catatan penting:

Eraser tidak benar-benar menghapus objek dari daftar `canvas.objects`. Eraser membuat objek baru berwarna putih di atas gambar lama. Secara visual terlihat seperti menghapus karena background canvas juga putih.

11. Fitur Fill

File:

- `app/tools/fill_tool.py`
- `app/algorithms/fill.py`

Fill digunakan untuk mengisi area tertutup. Algoritma yang digunakan adalah BFS Flood Fill.

Workflow fill:

1. Pengguna memilih tool Fill.
2. Pengguna klik area pada canvas.
3. Canvas saat ini dirender ke `QImage` sementara bernama `snapshot`.
4. Snapshot diberi background putih.
5. Semua objek canvas digambar ke snapshot.
6. Warna pixel pada titik klik dibaca sebagai `target_color`.
7. Warna pengisi diambil dari `canvas.stroke_color`.
8. Jika `target_color` sudah sama dengan warna fill, proses dihentikan.
9. Algoritma BFS mencari semua pixel yang warnanya sama dengan `target_color`.

10. Pixel hasil BFS digambar ke `fill_img`, yaitu `QImage` transparan.
11. `fill_img` disimpan sebagai `GraphicObject` baru.
12. Objek fill ditambahkan ke canvas.

Kenapa fill memakai snapshot?

Algoritma flood fill membutuhkan informasi warna pixel yang sudah tergambar. Karena objek pada canvas disimpan sebagai bentuk vektor, aplikasi perlu merender semuanya dulu ke gambar bitmap sementara. Setelah itu flood fill bisa bekerja pada level pixel.

Kenapa antialiasing snapshot dimatikan?

Di `FillTool`, render snapshot memakai:

```
painter.setRenderHint(QPainter.Antialiasing, False)
```

Tujuannya agar warna pixel lebih konsisten. Jika antialiasing aktif, tepi objek dapat memiliki warna campuran, sehingga flood fill bisa bocor atau berhenti di tempat yang tidak diharapkan.

12. Algoritma BFS Flood Fill

File: `app/algorithms/fill.py`

BFS Flood Fill mencari area terhubung berdasarkan warna pixel.

Input utama:

- `pixels`: fungsi untuk membaca warna pixel (`x`, `y`).
- `width`: lebar gambar.
- `height`: tinggi gambar.
- `x`, `y`: titik awal klik.
- `target_color`: warna yang ingin diganti.
- `fill_color`: warna pengganti.

Langkah algoritma:

1. Jika `target_color == fill_color`, kembalikan list kosong.
2. Buat queue menggunakan `deque`.
3. Masukkan titik awal ke queue.
4. Buat `visited` agar pixel tidak diproses berulang.
5. Selama queue tidak kosong:
 - Ambil pixel dari depan queue.
 - Jika di luar batas gambar, lewati.
 - Jika sudah dikunjungi, lewati.
 - Tandai pixel sebagai visited.
 - Jika warna pixel tidak sama dengan `target_color`, lewati.
 - Tambahkan pixel ke daftar hasil.
 - Tambahkan tetangga kanan, kiri, bawah, dan atas ke queue.

Tetangga yang diperiksa:

```
(x + 1, y)
(x - 1, y)
(x, y + 1)
(x, y - 1)
```

Jenis konektivitas ini disebut 4-connected flood fill.

Kompleksitas:

- Waktu: $O(W * H)$ pada kasus terburuk, ketika hampir seluruh canvas harus diperiksa.
- Memori: $O(W * H)$ untuk `visited` dan queue pada kasus terburuk.

13. Fitur Shape

File: `app/tools/shape_tool.py`

ShapeTool menangani pembuatan objek:

- Point
- Line
- Circle
- Ellipse
- Rectangle
- Triangle
- Trapezoid

Workflow umum shape:

1. Pengguna memilih shape dari dropdown.
2. Sidebar membuat `ShapeTool(canvas, kind)`.
3. Saat mouse ditekan, tool membuat `GraphicObject` sementara.
4. Saat mouse bergerak, titik akhir objek diperbarui sebagai preview.
5. Saat mouse dilepas, objek masuk ke `canvas.objects`.
6. Preview dihapus.
7. Canvas di-render ulang.

Untuk `point`:

1. Saat mouse ditekan, objek langsung dibuat.
2. Tidak perlu drag.
3. Objek langsung masuk ke canvas.

Untuk shape lain:

1. Titik awal adalah posisi mouse saat ditekan.
2. Titik akhir adalah posisi mouse saat dilepas.
3. Selama drag, `preview_object` digambar dengan alpha lebih transparan.

Untuk triangle dan trapezoid:

1. Pengguna melakukan drag dari titik awal ke titik akhir.
2. Dua titik drag tersebut membentuk bounding box.
3. Triangle dibuat dari tiga titik: puncak tengah atas, kiri bawah, dan kanan bawah.
4. Trapezoid dibuat dari empat titik: kiri atas yang diberi inset, kanan atas yang diberi inset, kanan bawah, dan kiri bawah.
5. Outline triangle dan trapezoid tidak digambar dengan `drawPolygon` biasa, tetapi digambar per sisi menggunakan algoritma garis aktif, yaitu DDA atau Bresenham.

14. Algoritma Garis

File: `app/algorithms/line.py`

CoretDraw menyediakan dua algoritma garis:

- Bresenham Line
- DDA Line

Pilihan algoritma disimpan pada `canvas.line_algo`, lalu ikut disimpan ke `GraphicObject` saat objek line dibuat.

Renderer memilih algoritma:

```
if obj.line_algo == "dda":
    gunakan dda_line
else:
    gunakan bresenham_line
```

14.1 DDA Line

DDA adalah singkatan dari Digital Differential Analyzer.

Ide utama:

1. Hitung selisih koordinat:

```
dx = x1 - x0
dy = y1 - y0
```

2. Tentukan jumlah langkah:

```
steps = max(abs(dx), abs(dy))
```

3. Hitung increment:

```
x_inc = dx / steps  
y_inc = dy / steps
```

4. Mulai dari (x_0, y_0) .
5. Setiap langkah, tambahkan x_inc dan y_inc .
6. Koordinat dibulatkan dengan $round$.

Kelebihan DDA:

- Mudah dipahami.
- Cocok untuk penjelasan dasar rasterisasi garis.

Kekurangan DDA:

- Menggunakan operasi floating point.
- Pembulatan dapat menghasilkan distribusi pixel yang kurang konsisten dibanding Bresenham.

Kompleksitas:

- Waktu: $O(n)$, dengan n adalah panjang garis dalam pixel.
- Memori: $O(n)$, karena fungsi mengembalikan list titik.

14.2 Bresenham Line

Bresenham Line menggambar garis menggunakan bilangan integer dan error term.

Ide utama:

1. Hitung:

```
dx = abs(x1 - x0)  
dy = abs(y1 - y0)
```

2. Tentukan arah gerak:

```
sx = 1 atau -1  
sy = 1 atau -1
```

3. Simpan error awal:

```
err = dx - dy
```

4. Selama titik belum sampai ke (x_1, y_1) , tambahkan pixel saat ini.
5. Hitung $e2 = 2 * err$.
6. Jika $e2 > -dy$, geser x dan kurangi error.

7. Jika $e2 < dx$, geser y dan tambah error.

Kelebihan Bresenham:

- Efisien.
- Tidak bergantung pada floating point untuk keputusan utama.
- Cocok untuk rasterisasi garis pada pixel grid.

Kompleksitas:

- Waktu: $O(n)$.
- Memori: $O(n)$.

15. Algoritma Lingkaran

File: `app/algorithms/circle.py`

CoretDraw menyediakan:

- Midpoint Circle
- Bresenham Circle

Renderer menghitung lingkaran dari dua titik:

- Titik pertama: pusat lingkaran.
- Titik kedua: titik tepi.

Radius dihitung dengan:

```
r = sqrt((x_edge - x_center)^2 + (y_edge - y_center)^2)
```

15.1 Simetri 8 Arah

Lingkaran memiliki simetri 8 arah. Jika satu titik (x, y) pada lingkaran diketahui, maka 7 titik lain dapat diperoleh dari simetri:

```
( cx + x, cy + y )  
( cx - x, cy + y )  
( cx + x, cy - y )  
( cx - x, cy - y )  
( cx + y, cy + x )  
( cx - y, cy + x )  
( cx + y, cy - x )  
( cx - y, cy - x )
```

Dengan simetri ini, algoritma hanya perlu menghitung satu bagian kecil lingkaran.

15.2 Midpoint Circle

Midpoint Circle menggunakan parameter keputusan untuk menentukan pixel berikutnya.

Inisialisasi:

```
x = 0
y = r
p = 1 - r
```

Setiap iterasi:

- Jika $p < 0$, pixel berikutnya bergerak ke arah timur.
- Jika $p \geq 0$, pixel berikutnya bergerak ke tenggara dan y dikurangi.

Update parameter:

```
Jika p < 0:
    p = p + 2*x + 1
Jika p >= 0:
    p = p + 2*(x - y) + 1
```

Loop berjalan selama $x < y$.

15.3 Bresenham Circle

Bresenham Circle mirip dengan midpoint circle, tetapi memakai decision variable:

```
d = 3 - 2*r
```

Setiap iterasi:

- Jika $d < 0$, pilih pixel timur.
- Jika $d \geq 0$, pilih pixel tenggara dan kurangi y .

Update:

```
Jika d < 0:
    d = d + 4*x + 6
Jika d >= 0:
    d = d + 4*(x - y) + 10
```

Kompleksitas algoritma lingkaran:

- Waktu: $O(r)$.
- Memori: $O(r)$, karena titik hasil disimpan dalam list.

16. Algoritma Elips

File: `app/algorithms/ellipse.py`

CoretDraw menyediakan:

- Midpoint Ellipse
- Bresenham Ellipse

Renderer membuat elips dari rectangle yang dibentuk oleh dua titik drag. Dari rectangle itu diperoleh:

```
cx = pusat rectangle x
cy = pusat rectangle y
rx = lebar rectangle / 2
ry = tinggi rectangle / 2
```

16.1 Simetri 4 Arah

Elips memiliki simetri 4 arah. Jika satu titik (x, y) diketahui, titik lain dapat dibuat:

```
(cx + x, cy + y)
(cx - x, cy + y)
(cx + x, cy - y)
(cx - x, cy - y)
```

16.2 Midpoint Ellipse

Midpoint Ellipse membagi proses ke dua region.

Region 1:

- Kemiringan kurva masih relatif landai.
- x bertambah lebih dominan.
- Loop berjalan selama:

```
2 * ry^2 * x < 2 * rx^2 * y
```

Region 2:

- Kemiringan kurva lebih curam.
- y berkurang lebih dominan.
- Loop berjalan selama:

```
y >= 0
```

Parameter keputusan region 1:

$$p1 = ry^2 - rx^2 * ry + 0.25 * rx^2$$

Parameter keputusan region 2:

$$p2 = ry^2 * (x + 0.5)^2 + rx^2 * (y - 1)^2 - rx^2 * ry^2$$

16.3 Bresenham Ellipse

Bresenham Ellipse juga dibagi menjadi region 1 dan region 2, tetapi memakai bentuk update integer yang lebih efisien.

Nilai yang digunakan:

```
rx2 = rx * rx
ry2 = ry * ry
two_rx2 = 2 * rx2
two_ry2 = 2 * ry2
```

Region 1 berjalan selama $dx < dy$, lalu region 2 berjalan selama $y \geq 0$.

Kompleksitas elips:

- Waktu: $O(rx + ry)$.
- Memori: $O(rx + ry)$.

17. Rendering Tiap Objek

File: `app/canvas/renderer.py`

17.1 Point

Point digambar sebagai ellipse kecil.

Radius:

```
r = max(3, obj.width)
```

17.2 Line

Line tidak digambar langsung dengan `drawLine`. Renderer memanggil algoritma garis terlebih dahulu untuk menghasilkan pixel, lalu setiap pixel digambar dengan `drawPoint`.

Workflow:

1. Ambil dua titik dari `obj.points`.
2. Pilih DDA atau Bresenham.
3. Dapatkan daftar pixel.
4. Gambar setiap pixel.

17.3 Circle

Circle memakai dua tahap:

1. Jika fill aktif, bagian dalam lingkaran digambar dengan `drawEllipse`.
2. Outline digambar pixel-per-pixel memakai Midpoint Circle atau Bresenham Circle.

17.4 Ellipse

Ellipse juga memakai dua tahap:

1. Jika fill aktif, bagian dalam elips digambar dengan `drawEllipse`.
2. Outline digambar pixel-per-pixel memakai Midpoint Ellipse atau Bresenham Ellipse.

17.5 Rectangle

Rectangle digambar dengan `painter.drawRect(rect)`.

Rectangle menggunakan dua titik drag yang dinormalisasi menjadi `QRectF`.

17.6 Triangle dan Trapezoid

Triangle dan trapezoid dibuat dari bounding box dua titik drag.

Triangle memakai titik:

```
top    = tengah atas bounding box
left   = kiri bawah bounding box
right  = kanan bawah bounding box
```

Trapezoid memakai titik:

```
top_left    = kiri atas + inset 25% lebar
top_right   = kanan atas - inset 25% lebar
bottom_right = kanan bawah
bottom_left  = kiri bawah
```

Jika fill aktif, area dalam shape digambar lebih dulu sebagai polygon tertutup. Setelah itu outline digambar edge-per-edge menggunakan algoritma garis aktif:

- Jika `obj.line_algo == "dda"`, sisi digambar dengan DDA.
- Jika tidak, sisi digambar dengan Bresenham.

Dengan pendekatan ini, triangle dan trapezoid tetap memanfaatkan algoritma garis yang sudah ada di project.

17.7 Brush dan Eraser

Brush dan eraser sama-sama digambar dengan:

```
painter.drawPath(obj.path)
```

Perbedaannya:

- Brush memakai warna stroke aktif.
- Eraser memakai warna putih.

17.8 Fill Image

Objek fill memiliki `image`. Renderer langsung menggambar image ke canvas:

```
painter.drawImage(0, 0, obj.image)
```

Karena fill disimpan sebagai bitmap overlay berukuran canvas, posisinya dimulai dari `(0, 0)`.

18. Warna dan Style

Warna utama:

- `stroke_color`: warna garis aktif.
- `fill_color`: warna isi aktif.

Saat objek dibuat, warna aktif di canvas disalin ke objek.

Jika pengguna memilih objek lalu mengubah warna:

1. Sidebar menyimpan state ke history.
2. Sidebar mengubah `stroke` atau `fill` pada semua objek terpilih.
3. Canvas di-update.

Fill color memiliki alpha 0 secara default, artinya transparan atau tidak ada fill.

19. Ukuran Garis dan Brush

Ukuran dikontrol oleh slider di sidebar.

Nilai disimpan pada:

```
canvas.line_width
```

Jika objek baru dibuat, `line_width` disalin ke `obj.width`.

Jika objek sedang dipilih lalu slider diubah:

1. State canvas disimpan ke history.
2. Width objek terpilih diperbarui.
3. Canvas di-render ulang.

20. Transformasi Objek

File:

- `app/ui/sidebar.py`
- `app/transforms/transformer.py`
- `app/canvas/renderer.py`

Transformasi yang tersedia di UI:

- Translate
- Rotate
- Scale
- Grayscale

20.1 Translate

Translate memindahkan objek sejauh `dx` dan `dy`.

Formula:

$$\begin{aligned}x' &= x + dx \\ y' &= y + dy\end{aligned}$$

Implementasi:

```
obj.move_by(dx, dy)
```

Untuk objek berbasis titik, semua titik digeser.

Untuk objek berbasis image, method `move_by` mencoba menggeser isi image dengan membuat image baru dan menggambar image lama pada offset `(dx, dy)`.

20.2 Rotate

Rotate menambah nilai `obj.rotation`.

Formula umum:

$$\begin{aligned}x' &= x \cos(a) - y \sin(a) \\y' &= x \sin(a) + y \cos(a)\end{aligned}$$

Dalam implementasi CoretDraw, titik objek tidak dihitung ulang secara manual. Rotasi diterapkan saat rendering memakai transformasi `QPainter`.

Workflow render rotasi:

1. Hitung pusat objek.
2. Painter digeser ke pusat objek.
3. Painter dirotasi sebesar `obj.rotation`.
4. Painter dikembalikan ke posisi semula.
5. Objek digambar.

Dengan pendekatan ini, data titik asli tetap sama, tetapi tampilan objek berputar.

20.3 Scale

Scale mengalikan faktor `scale_x` dan `scale_y`.

Formula:

$$\begin{aligned}x' &= x * sx \\y' &= y * sy\end{aligned}$$

Sama seperti rotate, skala diterapkan saat rendering melalui `QPainter`, bukan dengan mengubah langsung semua titik objek.

Workflow render scale:

1. Hitung pusat objek.
2. Painter digeser ke pusat objek.
3. Painter menerapkan `scale(scale_x, scale_y)`.
4. Painter dikembalikan.
5. Objek digambar.

20.4 Grayscale

Grayscale mengubah warna stroke dan fill menjadi abu-abu.

Formula luminansi:

$$\text{gray} = 0.299 * R + 0.587 * G + 0.114 * B$$

Setelah nilai gray dihitung:

```
R = gray
G = gray
B = gray
Alpha tetap
```

Formula ini mengikuti bobot luminansi umum ITU-R BT.601, yaitu hijau memiliki bobot paling besar karena mata manusia lebih sensitif terhadap hijau.

20.5 Reset Transform

Di dalam kode `Transformer` masih ada method `reset`, tetapi tombol Reset pada sidebar sudah dihilangkan. Method tersebut dapat mengembalikan:

```
rotation = 0
scale_x = 1
scale_y = 1
```

Saat ini fitur tersebut tidak tampil sebagai tombol di UI.

21. Undo dan Redo

File:

- `app/history/history_manager.py`
- `app/canvas/canvas_widget.py`

Undo/redo memakai dua stack:

- `_undo_stack`
- `_redo_stack`

Saat ada aksi baru:

1. State canvas saat ini dicopy dengan `clone_state()`.
2. State tersebut masuk ke undo stack.
3. Redo stack dibersihkan.
4. Jika undo stack lebih dari `max_size`, state paling lama dibuang.

21.1 clone_state

Canvas membuat salinan scene dengan:

```
[obj.clone() for obj in self.objects]
```

Setiap `GraphicObject` membuat salinan:

- titik,
- warna,
- ukuran,
- algoritma,
- transformasi,
- selected flag,
- image,
- path.

Hal ini penting agar undo tidak hanya menyimpan referensi objek yang sama.

21.2 Undo

Workflow undo:

1. Pengguna menekan **Ctrl + Z** atau menu Undo.
2. Canvas mengirim current state ke **history.undo(...)**.
3. Current state masuk ke redo stack.
4. State terakhir dari undo stack dikembalikan.
5. Canvas mengganti **objects** dengan state tersebut.

21.3 Redo

Workflow redo:

1. Pengguna menekan **Ctrl + Shift + Z** atau menu Redo.
2. Canvas mengirim current state ke **history.redo(...)**.
3. Current state masuk ke undo stack.
4. State terakhir dari redo stack dikembalikan.
5. Canvas memulihkan state tersebut.

Catatan:

Beberapa aksi sudah menyimpan history, misalnya membuat brush, eraser, fill, shape, clear, mengubah warna objek, mengubah ukuran objek, dan transformasi dari sidebar. Drag move pada SelectTool saat ini belum menyimpan state lama secara lengkap untuk undo.

22. Z-Order Objek

File: **app/ui/sidebar.py**

Z-order adalah urutan tumpukan objek. Objek yang berada di akhir list **canvas.objects** akan digambar terakhir, sehingga tampak berada di atas.

22.1 Bring Front

Workflow:

1. Ambil objek yang selected.
2. Ambil objek yang tidak selected.
3. Susun ulang list menjadi:

```
rest + selected
```

Karena selected berada di akhir list, objek tersebut digambar paling akhir dan terlihat di depan.

22.2 Send Back

Workflow:

1. Ambil objek yang selected.
2. Ambil objek yang tidak selected.
3. Susun ulang list menjadi:

```
selected + rest
```

Karena selected berada di awal list, objek tersebut digambar lebih dulu dan terlihat di belakang.

23. Delete Objek

File: `app/ui/sidebar.py`

Delete bisa dijalankan dari:

- Tombol Delete di sidebar.
- Menu `Edit > Delete`.
- Shortcut `Delete`.

Workflow:

1. Jika tidak ada objek terpilih, proses berhenti.
2. State canvas disimpan ke history.
3. Setiap objek terpilih dihapus dari `canvas.objects`.
4. `selected_objects` dikosongkan.
5. Canvas di-update.

24. Animasi

File: `app/animation/animator.py`

Animasi memakai `QTimer`. Timer memanggil method `_step()` setiap interval tertentu. Default interval adalah 30 ms.

Workflow:

1. Pengguna memilih menu Animation.
2. `Animator.start(mode)` dipanggil.
3. Animator menyimpan mode animasi.
4. Timer dimulai.
5. Setiap tick, `_step()` mengambil objek terpilih pertama dari canvas.

6. Objek diubah sesuai mode.
7. Canvas di-update.
8. Jika tidak ada objek terpilih, animasi berhenti.

Mode animasi:

24.1 Move

Objek digeser ke kanan:

```
obj.move_by(2, 0)
```

Jika objek melewati lebar canvas, objek digeser kembali ke kiri.

24.2 Rotate

Objek diputar:

```
obj.rotation += 3
```

Karena renderer memakai `QPainter.rotate`, objek tampak berputar saat canvas digambar ulang.

24.3 Scale

Animator memakai fungsi sinus agar skala berubah naik-turun secara halus.

Workflow:

1. `_angle` bertambah 5 derajat.
2. Faktor skala dihitung:

```
s = 1 + 0.01 * sin(angle)
```

3. `scale_x` dan `scale_y` dikalikan `s`.

24.4 Bounce

Objek digeser diagonal:

```
obj.move_by(3, 2)
```

Jika bounding box melewati batas kanan atau bawah canvas, objek digeser balik:

```
obj.move_by(-80, -60)
```

Animasi bounce ini masih sederhana, belum memakai kecepatan terpisah atau deteksi pantulan fisika penuh.

25. Export PNG

File: `app/ui/main_window.py`

Fitur export menyimpan tampilan canvas ke file PNG.

Workflow:

1. Pengguna memilih `File > Export PNG`.
2. Aplikasi membuka dialog `QFileDialog.getSaveFileName`.
3. Jika pengguna memilih path, aplikasi membuat `QImage` sebesar ukuran canvas.
4. Image diisi background putih.
5. Canvas dirender ke image dengan `self.canvas.render(p)`.
6. Painter ditutup.
7. Image disimpan ke path yang dipilih.
8. Status bar menampilkan lokasi file tersimpan.

26. Zoom dan Geser Canvas

File: `app/canvas/canvas_widget.py`

Canvas mendukung kontrol wheel:

26.1 Ctrl + Scroll

Digunakan untuk zoom.

Workflow:

1. Jika scroll ke atas, faktor zoom `1.15`.
2. Jika scroll ke bawah, faktor zoom `1 / 1.15`.
3. Nilai zoom dibatasi antara `0.1` dan `10.0`.
4. Canvas di-update.
5. Status bar menampilkan persen zoom.

26.2 Shift + Scroll

Digunakan untuk menggeser tampilan horizontal.

Workflow:

1. Jika scroll ke atas, `_offset_x` bertambah 20.
2. Jika scroll ke bawah, `_offset_x` berkurang 20.
3. Canvas di-update.

26.3 Penerapan di paintEvent

Saat menggambar:

```
painter.scale(_zoom, _zoom)
painter.translate(_offset_x, _offset_y)
```

Artinya semua objek digambar berdasarkan transformasi tampilan tersebut.

27. Status Bar

File:

- `app/ui/main_window.py`
- `app/canvas/canvas_widget.py`
- `app/ui/sidebar.py`

Status bar digunakan untuk:

- Menampilkan posisi kursor.
- Menampilkan status awal aplikasi.
- Menampilkan info zoom.
- Menampilkan info algoritma aktif.
- Menampilkan path file setelah export PNG.

Saat mouse bergerak di canvas, canvas memanggil:

```
parent_window.update_coords(x, y)
```

Lalu status bar menampilkan:

```
x=..., y=...
```

28. Shortcut

Shortcut yang tersedia:

- `Ctrl + Z`: Undo.
- `Ctrl + Shift + Z`: Redo.
- `Delete`: Hapus objek terpilih.
- `Ctrl + Scroll`: Zoom canvas.
- `Shift + Scroll`: Geser canvas horizontal.

29. Workflow Lengkap Menggambar Objek

Contoh workflow menggambar garis:

1. Pengguna memilih shape `Line`.

2. Sidebar membuat `ShapeTool(canvas, "line")`.
3. Pengguna memilih algoritma garis, misalnya Bresenham.
4. Pengguna menekan mouse pada canvas.
5. `ShapeTool.on_press` membuat `GraphicObject` sementara.
6. Pengguna drag mouse.
7. `ShapeTool.on_move` memperbarui titik akhir preview.
8. Canvas menggambar preview dengan alpha lebih transparan.
9. Pengguna melepas mouse.
10. State lama disimpan ke history.
11. Objek masuk ke `canvas.objects`.
12. Canvas memanggil `update`.
13. Renderer membaca `obj.kind == "line"`.
14. Renderer memilih algoritma garis dari `obj.line_algo`.
15. Algoritma menghasilkan daftar pixel.
16. Renderer menggambar pixel satu per satu.

Contoh workflow menggambar lingkaran:

1. Pengguna memilih shape `Circle`.
2. Titik tekan mouse menjadi pusat lingkaran.
3. Titik lepas mouse menjadi titik tepi.
4. Radius dihitung dengan jarak Euclidean.
5. Renderer memilih Midpoint Circle atau Bresenham Circle.
6. Algoritma menghasilkan pixel outline memakai simetri 8 arah.
7. Jika fill aktif, bagian dalam digambar dengan `drawEllipse`.
8. Outline digambar pixel-per-pixel.

Contoh workflow fill:

1. Pengguna memilih Fill.
2. Pengguna memilih warna stroke yang akan menjadi warna isi.
3. Pengguna klik area canvas.
4. Scene dirender ke snapshot bitmap.
5. Warna pixel target dibaca.
6. BFS mencari area yang warnanya sama.
7. Hasil BFS dibuat menjadi image transparan.
8. Image transparan ditambahkan sebagai objek baru di canvas.

30. Alur Data Objek dari Dibuat sampai Dirender

Alur umum:

```
Input pengguna
-> Tool aktif
-> GraphicObject
-> canvas.objects
-> CanvasWidget.paintEvent
-> Renderer.draw
```

```
-> QPainter
-> Tampilan di layar
```

Objek tidak langsung menjadi pixel permanen di canvas, kecuali hasil fill yang memang disimpan sebagai **QImage**. Sebagian besar objek tetap disimpan sebagai data bentuk, sehingga bisa dipilih, dipindahkan, diubah warna, dan ditransformasi.

31. Keterbatasan Implementasi Saat Ini

Beberapa hal yang perlu diketahui:

1. Eraser bekerja secara visual dengan menggambar putih, bukan menghapus objek asli.
2. Drag move pada SelectTool belum menyimpan history undo secara lengkap.
3. Hit test memakai bounding box, belum presisi mengikuti bentuk asli objek.
4. Fill bergantung pada warna pixel snapshot, sehingga celah kecil pada outline dapat membuat fill bocor.
5. Objek fill berbasis image diperlakukan berbeda dari objek vektor.
6. Animasi bounce masih sederhana dan belum memakai model kecepatan pantulan yang lengkap.
7. Transformasi rotate dan scale diterapkan saat rendering, sehingga data titik asli tidak berubah.

32. Ringkasan Algoritma yang Digunakan

Fitur	Algoritma / Teknik
Garis	DDA, Bresenham Line
Lingkaran	Midpoint Circle, Bresenham Circle
Elips	Midpoint Ellipse, Bresenham Ellipse
Triangle/Trapezoid	Bounding box + sisi-sisi DDA/Bresenham
Fill	BFS Flood Fill 4-connected
Brush	QPainterPath
Eraser	QPainterPath warna putih
Select	Bounding box hit test dan rectangle intersection
Transformasi	Translate titik, rotate dan scale via QPainter transform
Grayscale	Luminance formula $0.299R + 0.587G + 0.114B$
Undo/Redo	Stack state clone
Animasi	QTimer dengan update berkala
Export PNG	Render canvas ke QImage

33. Kesimpulan

CoretDraw dibangun dengan pendekatan modular. Canvas bertugas menerima event dan menyimpan state, tool bertugas menerjemahkan input pengguna menjadi perubahan objek, renderer bertugas menggambar objek, sedangkan algoritma grafika ditempatkan terpisah agar mudah dipelajari.

Project ini cocok sebagai media pembelajaran grafika komputer karena memperlihatkan hubungan antara:

- input mouse,
- struktur data objek,
- algoritma rasterisasi,
- rendering dengan QPainter,
- transformasi 2D,
- flood fill,
- undo/redo,
- dan animasi berbasis timer.

Dengan membaca kode dan dokumen ini, alur kerja setiap fitur dapat ditelusuri dari interaksi pengguna sampai hasil akhir yang tampil pada canvas.